Comprehensive Theory of Single-Cycle Processor Design and the ToyRISC ISA

Part 3: Simulator Architecture, Code Walkthrough, and Test Programs

Computer Architecture Laboratory – Assignment 3

February 15, 2026

Contents

1 Simulator Architecture Overview **2**
- 1.1 Package Structure . . . 2
- 1.2 Execution Flow . . . 2

2 Detailed Code Walkthrough **3**
- 2.1 Main.java – Entry Point . . . 3
- 2.2 Processor.java – The Processor Model . . . 3
- 2.3 Simulator.java – Simulation Control . . . 4
- 2.4 Latch Types – Data Flow Between Stages . . . 5
  - 2.4.1 IF_EnableLatchType . . . 5
  - 2.4.2 IF_OF_LatchType . . . 6
  - 2.4.3 OF_EX_LatchType (Must be Extended) . . . 6
  - 2.4.4 EX_MA_LatchType (Must be Extended) . . . 6
  - 2.4.5 EX_IF_LatchType (Must be Extended) . . . 6
  - 2.4.6 MA_RW_LatchType (Must be Extended) . . . 7

3 Configuration System **7**
- 3.1 config.xml . . . 7
- 3.2 Configuration Parameters Explained . . . 7

4 Test Program Analysis **8**
- 4.1 Even or Odd (`evenorodd.asm`) . . . 8
- 4.2 Fibonacci Sequence (`fibonacci.asm`) . . . 9
- 4.3 Palindrome Check (`palindrome.asm`) . . . 10
- 4.4 Prime Check (`prime.asm`) . . . 10
- 4.5 Descending Sort (`descending.asm`) . . . 11

5 Statistics and Performance Analysis **12**
5.1 Statistics Collection 12
5.2 Counting Instructions 12
5.3 CPI Analysis 12
5.4 Performance Comparison with Pipelined Processors 12

6 Verification and Testing **13**
6.1 Hash-Based Verification 13
6.2 test_zip.py – Automated Testing 13
6.3 Build System 13

7 Implementation Checklist **14**

8 Common Pitfalls and Debugging Tips **16**

9 Advanced Topics – Preview for Future Labs **16**
9.1 Pipelining 16
9.2 Cache Memory 17
9.3 Multi-Cycle Functional Units 17

10 Summary **17**

1 Simulator Architecture Overview

The ToyRISC simulator is a Java-based architectural simulator that models a single-cycle processor. It is organized into four packages reflecting a clean separation of concerns.

1.1 Package Structure

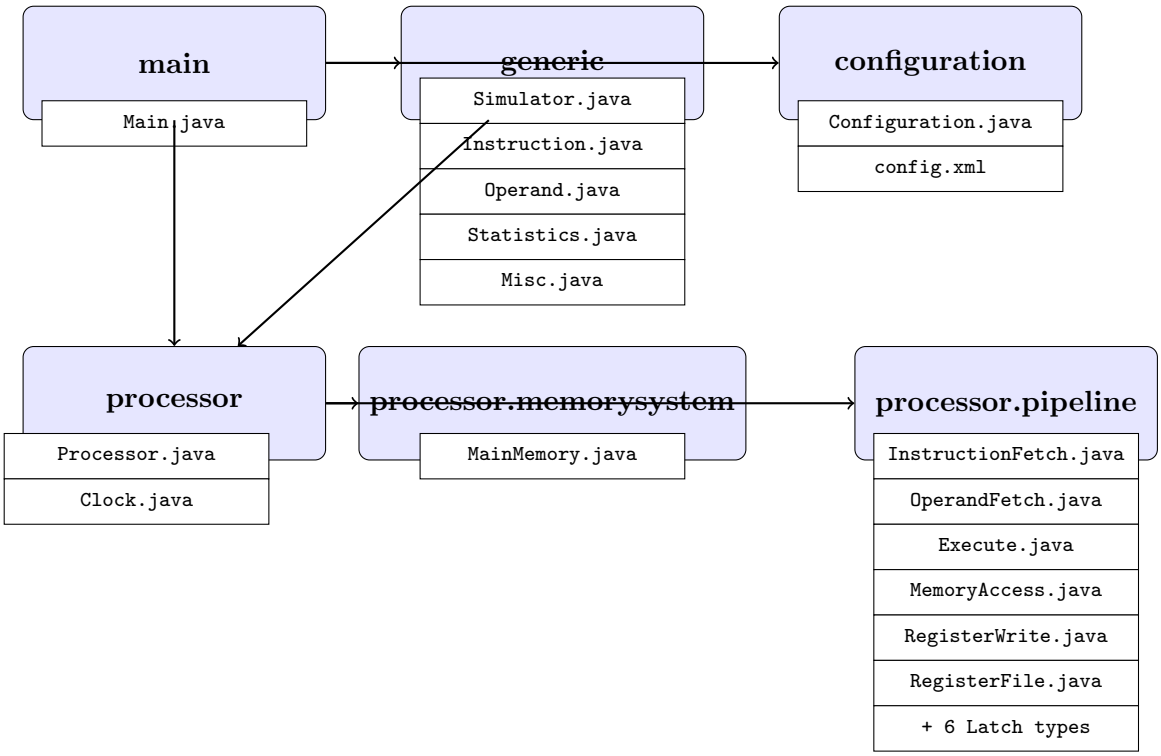

Figure 1: Package dependency diagram

1.2 Execution Flow

The program execution follows this sequence:

1. `Main.main()` parses command-line arguments (config file, stats file, object file).
2. `Configuration.parseConfigurationFile()` reads processor parameters from XML.
3. A `Processor` object is created, which instantiates all pipeline stages, latches, register file, and memory.
4. `Simulator.setupSimulation()` loads the object file into memory and initializes registers.
5. `Simulator.simulate()` runs the main simulation loop: IF → OF → EX → MA → RW, repeating until `end` is encountered.
6. Final processor state is printed (registers and memory).
7. Statistics are written to the output file.
8. A hash of the final processor state is computed and printed (for verification).

2 Detailed Code Walkthrough

2.1 Main.java – Entry Point

```
public class Main {
    public static void main(String[] args) {
        // args[0] = config file path
        // args[1] = statistics output file path
        // args[2] = object file path

        Configuration.parseConfiguratioFile(args[0]);
        Processor processor = new Processor();

        Simulator.setupSimulation(args[2], processor);
        Simulator.simulate();

        processor.printState(0, 30);
        Statistics.printStatistics(args[1]);

        System.out.println("Hash of the Processor State = "
            + getHashCode(processor.getRegisterFile(),
                          processor.getMainMemory()));
    }

    static int getHashCode(RegisterFile regState,
                           MainMemory memState) {
        ArrayList<Integer> hash = new ArrayList<>();
        hash.add(regState.getProgramCounter());
        for (int i = 0; i < 32; i++)
            hash.add(regState.getValue(i));
        for (int i = 0; i < 65536; i++)
            hash.add(memState.getWord(i));
        return hash.hashCode();
    }
}
```

Listing 1: Main.java

Key Points:

- The hash function incorporates the **entire processor state**: PC, all 32 registers, and all 65536 memory words. This ensures correctness verification is comprehensive.
- The hash uses Java's `ArrayList.hashCode()`, which computes a deterministic hash based on all elements.

2.2 Processor.java – The Processor Model

The `Processor` class is the central coordinator. It owns all hardware components and wires them together.

```
public class Processor {
    RegisterFile registerFile;
    MainMemory mainMemory;

    // Inter-stage latches
    IF_EnableLatchType IF_EnableLatch;
```

```
    IF_OF_LatchType IF_OF_Latch;
    OF_EX_LatchType OF_EX_Latch;
    EX_MA_LatchType EX_MA_Latch;
    EX_IF_LatchType EX_IF_Latch;  // feedback latch
    MA_RW_LatchType MA_RW_Latch;

    // Pipeline stages
    InstructionFetch IFUnit;
    OperandFetch OFUnit;
    Execute EXUnit;
    MemoryAccess MAUnit;
    RegisterWrite RWUnit;

    public Processor() {
        registerFile = new RegisterFile();
        mainMemory = new MainMemory();

        // Create latches
        IF_EnableLatch = new IF_EnableLatchType();
        IF_OF_Latch = new IF_OF_LatchType();
        OF_EX_Latch = new OF_EX_LatchType();
        EX_MA_Latch = new EX_MA_LatchType();
        EX_IF_Latch = new EX_IF_LatchType();
        MA_RW_Latch = new MA_RW_LatchType();

        // Create stages, passing shared latches
        IFUnit = new InstructionFetch(this,
            IF_EnableLatch, IF_OF_Latch, EX_IF_Latch);
        OFUnit = new OperandFetch(this,
            IF_OF_Latch, OF_EX_Latch);
        EXUnit = new Execute(this,
            OF_EX_Latch, EX_MA_Latch, EX_IF_Latch);
        MAUnit = new MemoryAccess(this,
            EX_MA_Latch, MA_RW_Latch);
        RWUnit = new RegisterWrite(this,
            MA_RW_Latch, IF_EnableLatch);
    }
}
```

Listing 2: Processor.java – Constructor

Design Pattern – Dependency Injection: Each pipeline stage receives references to its input and output latches through the constructor. This is a form of dependency injection that:

- Decouples stages from each other (no stage directly references another stage).
- Makes the data flow explicit (each stage knows only about its latches).
- Facilitates testing (stages can be tested independently with mock latches).

The EX_IF Feedback Latch: Notice that the `EX_IF_Latch` connects the Execute stage back to the Instruction Fetch stage. This is essential for control flow instructions (branches and jumps) that need to redirect the PC.

2.3 Simulator.java – Simulation Control

```java
public class Simulator {
    static Processor processor;
    static boolean simulationComplete;

    public static void setupSimulation(String objFile,
                                       Processor p) {
        Simulator.processor = p;
        loadProgram(objFile);
        simulationComplete = false;
    }

    static void loadProgram(String objFile) {
        // TODO: Load object file into memory
        // 1. Read first int = PC
        // 2. Read remaining ints into memory[0], memory[1], ...
        // 3. Set PC, x0=0, x1=65535, x2=65535
    }

    public static void simulate() {
        while (simulationComplete == false) {
            processor.getIFUnit().performIF();
            Clock.incrementClock();
            processor.getOFUnit().performOF();
            Clock.incrementClock();
            processor.getEXUnit().performEX();
            Clock.incrementClock();
            processor.getMAUnit().performMA();
            Clock.incrementClock();
            processor.getRWUnit().performRW();
            Clock.incrementClock();
        }
        // TODO: Set statistics
    }
}
```

Listing 3: Simulator.java

Simulation Loop Analysis:

- Each iteration processes **one complete instruction** through all 5 stages.
- The clock increments 5 times per instruction (once per stage).
- Total cycles = 5 × number of instructions executed.
- The loop terminates when `simulationComplete` is set to `true` by the RW stage processing an `end` instruction.

2.4 Latch Types – Data Flow Between Stages

2.4.1 IF_EnableLatchType

Controls whether the IF stage should execute. Initialized to `true`. The RW stage re-enables it after completing an instruction.

2.4.2 IF_OF_LatchType

Carries the 32-bit instruction word from IF to OF. The skeleton provides this:

```
public class IF_OF_LatchType {
    boolean OF_enable;
    int instruction;   // 32-bit instruction word
}
```

2.4.3 OF_EX_LatchType (Must be Extended)

The provided skeleton only has an `EX_enable` flag. You must add fields to carry all decoded information:

```
public class OF_EX_LatchType {
    boolean EX_enable;

    // Decoded instruction info
    Instruction instruction;   // the full instruction object
    int operand1;              // value of source register 1
    int operand2;              // value of src reg 2 or immediate
    int destinationRegister;   // destination register number
    // ... any other fields needed
}
```

Listing 4: Suggested OF_EX_LatchType fields

2.4.4 EX_MA_LatchType (Must be Extended)

Must carry the ALU result, effective address, instruction type, and data:

```
public class EX_MA_LatchType {
    boolean MA_enable;

    Instruction instruction;
    int aluResult;
    // For store: the value to store
    // For load: the effective address
    // etc.
}
```

Listing 5: Suggested EX_MA_LatchType fields

2.4.5 EX_IF_LatchType (Must be Extended)

Must communicate branch/jump decisions back to IF:

```
public class EX_IF_LatchType {
    boolean isBranchTaken;
    int branchPC;   // new PC value if branch taken
}
```

Listing 6: Suggested EX_IF_LatchType fields

2.4.6 MA_RW_LatchType (Must be Extended)

Must carry the result to write back and which register to write to:

```
public class MA_RW_LatchType {
    boolean RW_enable;

    Instruction instruction;
    int result;     // ALU result or loaded data
    int destinationRegister;
}
```

Listing 7: Suggested MA_RW_LatchType fields

3 Configuration System

3.1 config.xml

The configuration file defines hardware parameters using XML:

Listing 8: config.xml

```
<Configuration>
    <FunctionalUnits>
        <ALU>
            <Count>2</Count>
            <Latency>1</Latency>
            <ReciprocalOfThroughput>1</ReciprocalOfThroughput>
        </ALU>
        <Multiplier>
            <Count>1</Count>
            <Latency>4</Latency>
        </Multiplier>
        <Divider>
            <Count>1</Count>
            <Latency>10</Latency>
        </Divider>
    </FunctionalUnits>

    <L1iCache>
        <NumberOfLines>256</NumberOfLines>
        <Latency>2</Latency>
        <Associativity>4</Associativity>
        <ReplacementPolicy>LRU</ReplacementPolicy>
    </L1iCache>
    <!-- L1dCache, L2Cache similar -->

    <MainMemoryLatency>40</MainMemoryLatency>
</Configuration>
```

3.2 Configuration Parameters Explained

Parameter	Value	Meaning

`ALU.Count`	2	Number of ALU functional units
`ALU.Latency`	1	Cycles to complete an ALU operation
`ALU.ReciprocalOfThroughput`	1	New operation every 1 cycle
`Multiplier.Count`	1	One multiplier unit
`Multiplier.Latency`	4	4 cycles per multiplication
`Divider.Count`	1	One divider unit
`Divider.Latency`	10	10 cycles per division
`L1i.NumberOfLines`	256	256 cache lines in L1 instruction cache
`L1i.Latency`	2	2-cycle hit latency
`L1i.Associativity`	4	4-way set-associative
`L1i.ReplacementPolicy`	LRU	Least Recently Used eviction
`MainMemoryLatency`	40	40-cycle memory access latency

Note: These parameters are not used in Lab 3 (single-cycle processor). They will be used in future labs when implementing pipelined processors with caches and multi-cycle functional units.

4 Test Program Analysis

4.1 Even or Odd (`evenorodd.asm`)

```
    .data
n:
    11
    .text
main:
    load %x0, $n, %x3       ; x3 = memory[0] = 11
    divi %x3, 2, %x3        ; x3 = 11/2 = 5, x31 = 11%2 = 1
    beq %x0, %x31, even     ; if x31 == 0, go to even
    sub %x10, %x10, %x10    ; x10 = 0
    addi %x10, 1, %x10      ; x10 = 1 (odd)
    end
even:
    sub %x10, $x10, %x10    ; x10 = 0
    subi %x10, 1, %x10      ; x10 = -1 (even)
    end
```

Listing 9: evenorodd.asm

Algorithm: Divides n by 2 using `divi`. The remainder is automatically stored in `x31`. If the remainder (`x31`) equals 0, the number is even; otherwise odd.

Result: For $n = 11$: remainder is $1 \neq 0$, so `x10 = 1` (odd).

Key insight: This program exploits the `x31` remainder feature of the `div/divi` instructions.

Execution trace:

1. `load`: x3 = 11
2. `divi`: x3 = 5, x31 = 1

3. `beq`: Compare x31 (=1) with x0 (=0). $1 \neq 0$, so branch NOT taken.
4. `sub`: x10 = 0
5. `addi`: x10 = 1
6. `end`: Simulation terminates.

Total instructions: 6.

4.2 Fibonacci Sequence (`fibonacci.asm`)

```
.data
n:
    10
    .text
main:
    addi %x0, 0, %x3        ; x3 = 0 (F(0))
    addi %x0, 1, %x4        ; x4 = 1 (F(1))
    add %x3, %x4, %x5       ; x5 = F(0)+F(1) = 1
    load %x0, $n, %x6       ; x6 = 10 (number count)
    addi %x0, 65535, %x7     ; x7 = 65535 (store addr)
    addi %x0, 0, %x8        ; x8 = 0 (counter)
    store %x3, 0, %x7       ; mem[65535] = F(0) = 0
    subi %x7, 1, %x7        ; x7 = 65534
    addi %x8, 1, %x8        ; x8 = 1
    store %x4, 0, %x7       ; mem[65534] = F(1) = 1
    subi %x7, 1, %x7        ; x7 = 65533
    addi %x8, 1, %x8        ; x8 = 2
for:
    blt %x8, %x6, loop      ; if counter < n, continue
    end
loop:
    add %x3, %x4, %x5       ; x5 = F(i-2) + F(i-1)
    store %x5, 0, %x7       ; mem[x7] = F(i)
    subi %x7, 1, %x7        ; x7--
    addi %x8, 1, %x8        ; counter++
    add %x0, %x4, %x3       ; x3 = x4 (shift window)
    add %x0, %x5, %x4       ; x4 = x5 (shift window)
    jmp for                  ; back to check
```

Listing 10: fibonacci.asm

Algorithm: Computes the first n Fibonacci numbers using an iterative approach and stores them in memory at addresses starting from 65535 downward.

Variables:

- `x3` = F(i-2), `x4` = F(i-1), `x5` = F(i)
- `x6` = n (target count), `x7` = memory store address, `x8` = counter

Loop invariant: At the start of each iteration of `loop`, `x3` = F(i-2), `x4` = F(i-1).

Memory after execution ($n = 10$):

- mem[65535] = 0, mem[65534] = 1, mem[65533] = 1, mem[65532] = 2, mem[65531] = 3,
- mem[65530] = 5, mem[65529] = 8, mem[65528] = 13, mem[65527] = 21, mem[65526] = 34

4.3 Palindrome Check (palindrome.asm)

```
.data
a:
    4567654
    .text
main:
    load %x0, $a, %x3       ; x3 = 4567654
    sub %x7, %x7, %x7       ; x7 = 0 (reversed number)
loop:
    divi %x3, 10, %x4       ; x4 = x3/10, x31 = x3%10
    addi %x31, 0, %x30      ; x30 = last digit
    muli %x7, 10, %x7       ; x7 = x7 * 10
    add %x7, %x30, %x7      ; x7 = x7 + last digit
    divi %x3, 10, %x3       ; x3 = x3 / 10
    bgt %x3, %x0, loop      ; if x3 > 0, continue
    load %x0, $a, %x5       ; x5 = original number
    beq %x5, %x7, palindrome  ; if reversed == original
    sub %x10, %x10, %x10
    subi %x10, 1, %x10      ; x10 = -1 (not palindrome)
    end
palindrome:
    sub %x10, %x10, %x10
    addi %x10, 1, %x10      ; x10 = 1 (is palindrome)
    end
```

Listing 11: palindrome.asm

Algorithm: Reverses the digits of the number by repeatedly extracting the last digit (using `divi` with remainder in `x31`) and building the reversed number. Then compares the reversed number with the original.

Key operations: Uses `divi` for both quotient (integer division by 10) and remainder (modulo 10, stored in `x31`), and `muli` for multiplication by 10.

Trace for 4567654:

Iteration	x3	x31 (digit)	x7 (reversed)
1	456765	4	4
2	45676	5	45
3	4567	6	456
4	456	7	4567
5	45	6	45676
6	4	5	456765
7	0	4	4567654

Reversed = 4567654 = Original. `x10 = 1` (palindrome).

4.4 Prime Check (prime.asm)

```
.data
a:
    11
    .text
main:
    load %x0, $a, %x3       ; x3 = 11 (number to check)
```

```
    sub %x4, %x4, %x4
    divi %x3, 2, %x4        ; x4 = 11/2 = 5 (upper bound)
    sub %x6, %x6, %x6
    addi %x6, 2, %x6        ; x6 = 2 (divisor, start from 2)
for:
    bgt %x6, %x4, prime     ; if divisor > n/2, it's prime
    div %x3, %x6, %x7       ; x7 = n/divisor, x31 = n%divisor
    beq %x0, %x31, notprime ; if remainder==0, not prime
    addi %x6, 1, %x6        ; divisor++
    jmp for
prime:
    sub %x10, %x10, %x10
    addi %x10, 1, %x10      ; x10 = 1 (prime)
    end
notprime:
    sub %x10, %x10, %x10
    subi %x10, 1, %x10      ; x10 = -1 (not prime)
    end
```

Listing 12: prime.asm

Algorithm: Trial division. Tests all divisors from 2 to $n/2$. If any divides n evenly (remainder $= 0$), n is not prime.

Complexity: $O(n/2)$ divisions.

For $n = 11$: Tests $d = 2, 3, 4, 5$. None divide 11 evenly. Result: `x10 = 1` (prime).

4.5 Descending Sort (`descending.asm`)

```
.data
a:
    40  20  50  60  80  30  10  70
n:
    8
    .text
main:
    sub %x3, %x3, %x3       ; x3 = 0 (outer index i)
    sub %x4, %x4, %x4       ; x4 = 0 (inner index j)
    load %x0, $n, %x8       ; x8 = 8 (array size)
outerloop:
    blt %x3, %x8, innerloop ; if i < n, continue
    end
    addi %x3, 1, %x4
innerloop:
    addi %x3, 1, %x4        ; j = i + 1
innerloopz:
    blt %x4, %x8, swap      ; if j < n, compare
    addi %x3, 1, %x3        ; i++  (typo: %3 should be %x3)
    jmp outerloop
swap:
    load %x3, $a, %x5       ; x5 = a[i]
    load %x4, $a, %x6       ; x6 = a[j]
    blt %x5, %x6, exchange  ; if a[i] < a[j], swap (descending)
    addi %x4, 1, %x4        ; j++
    jmp innerloopz
exchange:
    sub %x7, %x7, %x7
    add %x0, %x5, %x7       ; x7 = a[i] (temp)
```

```
    store %x6, 0, %x3       ; a[i] = a[j]  (using $a base=0)
    store %x7, 0, %x4       ; a[j] = temp
    addi %x4, 1, %x4        ; j++
    jmp innerloopz
```

Listing 13: descending.asm – Bubble Sort

Algorithm: Selection/bubble sort in descending order. For each position i, finds if any element $a[j > i]$ is greater than $a[i]$ and swaps them.

Result: Array sorted in descending order: 80, 70, 60, 50, 40, 30, 20, 10.

Note: The `store` instructions use the index registers directly as addresses because the array starts at address 0 (the label `a` points to address 0, so `$a = 0`, and `mem[x3 + 0] = mem[i]`).

5 Statistics and Performance Analysis

5.1 Statistics Collection

The `Statistics.java` class tracks:

- **numberOfInstructions**: Total instructions executed (count each instruction as it passes through the pipeline).
- **numberOfCycles**: Total clock ticks consumed. In the single-cycle model, this equals 5 × numberOfInstructions.

5.2 Counting Instructions

The simplest approach: increment a counter each time an instruction completes the RW stage (i.e., each iteration of the simulation loop).

```
// At the end of simulate():
Statistics.setNumberOfInstructions(instructionCount);
Statistics.setNumberOfCycles(5 * instructionCount);
```

5.3 CPI Analysis

For the single-cycle processor:

$$\text{CPI} = \frac{\text{Total Cycles}}{\text{Total Instructions}} = \frac{5N}{N} = 5$$

Every instruction takes exactly 5 clock ticks. This is optimal in terms of regularity but suboptimal in terms of performance.

5.4 Performance Comparison with Pipelined Processors

In future labs, pipelined execution will overlap stages of different instructions:

	Cycle 1	2	3	4	5	6	7	8	9
Single-Cycle (Lab 3):									
Instr 1	IF	OF	EX	MA	RW				
Instr 2						IF	OF	EX	MA
Pipelined (Future Labs):									
Instr 1	IF	OF	EX	MA	RW				
Instr 2		IF	OF	EX	MA	RW			
Instr 3			IF	OF	EX	MA	RW		

A pipelined processor achieves CPI ≈ 1 (one instruction per clock cycle) in the ideal case.

6 Verification and Testing

6.1 Hash-Based Verification

The simulator computes a hash of the entire processor state (PC + 32 registers + 65536 memory words) at the end of execution. This hash is compared against expected values provided in `.expected` files.

Test Case	Expected Hash
fibonacci	−1518357572
descending	255541867
evenorodd	−224294686

6.2 test_zip.py – Automated Testing

The `test_zip.py` script automates testing:

1. Extracts the submitted zip file.
2. Builds the project using `ant make-jar` (compiles Java and creates `simulator.jar`).
3. For each `.out` file in `test_cases/`: runs the simulator with the corresponding object file.
4. Compares the output hash with the expected hash from `.expected` files.
5. Reports PASS/fail for each test case and total score.

6.3 Build System

The `build.xml` is an Apache Ant build file:

- `ant build`: Compiles all Java source to `bin/` directory.
- `ant make-jar`: Creates `jars/simulator.jar` with main class `main.Main`.
- `ant clean`: Removes the `bin/` directory.

Running the simulator:

```
java -jar jars/simulator.jar \
    src/configuration/config.xml \
    output.stat \
    test_cases/fibonacci.out
```

7 Implementation Checklist

Here is a comprehensive checklist of what must be implemented:

Implementation Checklist

1. **loadProgram()** in `Simulator.java`:
   - Read object file
   - Load data and instructions into memory
   - Set PC to first instruction address
   - Initialize x0=0, x1=65535, x2=65535
2. **Extend latch types**:
   - `OF_EX_LatchType`: Add decoded instruction fields
   - `EX_MA_LatchType`: Add ALU result, instruction info
   - `EX_IF_LatchType`: Add branch decision and target PC
   - `MA_RW_LatchType`: Add result and destination register
3. **performOF()** in `OperandFetch.java`:
   - Extract opcode (bits 27–31)
   - Determine format (R3, R2I, or RI)
   - Extract register numbers and/or immediate
   - Sign-extend immediate values
   - Read register values from register file
   - Populate OF/EX latch
4. **performEX()** in `Execute.java`:
   - Implement all 30 operations (ALU, branches, jumps)
   - Handle x31 overflow/remainder/shifted-bits
   - Compute branch targets for control flow
   - Update PC via EX/IF latch for taken branches
   - Populate EX/MA latch
5. **performMA()** in `MemoryAccess.java`:
   - For `load`: read memory at effective address
   - For `store`: write value to memory at effective address
   - For other instructions: pass through
   - Populate MA/RW latch
6. **performRW()** in `RegisterWrite.java`:
   - Write result to destination register
   - Handle `end`: call `Simulator.setSimulationComplete(true)`
   - Re-enable IF stage
7. **Set statistics** in `Simulator.simulate()`:
   - Count instructions executed
   - Count cycles (5 × instructions)

8 Common Pitfalls and Debugging Tips

Common Pitfalls

1. **Wrong bit extraction**: Use `>>>` (unsigned shift) instead of `>>` (signed shift) when extracting opcodes and register numbers.
2. **Forgetting sign extension**: The 17-bit and 22-bit immediates are signed and must be sign-extended to 32 bits.
3. **Branch operand confusion**: For conditional branches, the ISA places the compared registers in sourceOperand1 and sourceOperand2, and the offset in destinationOperand. This is different from the assembly syntax.
4. **PC increment timing**: PC is incremented in the IF stage. When computing branch targets in EX, remember that PC already points to the *next* instruction.
5. **Store semantics**: In `store`, the address is `rd + imm` (not rs1 + imm, which is used for `load`).
6. **x0 immutability**: Register x0 must always be 0. If an instruction has rd = x0, the write should be discarded (or x0 should be reset to 0).
7. **x31 side effects**: `mul`, `div`, and all shift instructions modify x31 as a side effect. Don't forget to update x31 in the RW stage.
8. **End instruction flow**: The `end` instruction must pass through all 5 stages before the simulation terminates. Don't terminate early in the EX or MA stage.
9. **Object file format**: The first integer in the object file is the PC start address, *not* a data value. All subsequent values are loaded starting at memory address 0.

9 Advanced Topics – Preview for Future Labs

9.1 Pipelining

In a pipelined processor, multiple instructions are in different stages simultaneously. This increases throughput but introduces hazards:

- **Data Hazards**: An instruction depends on the result of a previous instruction that hasn't completed yet. Solutions: forwarding (bypassing), stalling (pipeline bubbles).
- **Control Hazards**: A branch instruction changes the PC, but subsequent instructions may have already been fetched. Solutions: branch prediction, pipeline flushing, delayed branches.
- **Structural Hazards**: Two instructions need the same hardware resource simultaneously. Solutions: resource duplication, stalling.

9.2 Cache Memory

The configuration file defines L1i, L1d, and L2 caches. Key concepts:

- **Set-Associative Mapping**: Each memory block maps to a specific set, and within a set there are k ways (4-way in config).
- **LRU Replacement**: When a cache set is full, the least recently used block is evicted.
- **Hit Ratio**: Fraction of accesses found in cache. Higher is better.
- **Miss Penalty**: Time to fetch from next level ($L1 \to L2 \to$ Main Memory).

9.3 Multi-Cycle Functional Units

The configuration specifies different latencies for ALU (1), multiplier (4), and divider (10). In future labs, these will create structural hazards if multiple instructions need the same unit simultaneously.

10 Summary

This three-part document has covered:

Part 1 Foundations of computer architecture: Von Neumann model, RISC vs CISC, registers, memory, single-cycle processor model, pipeline stages, inter-stage latches, control flow, function calling conventions, number representation.

Part 2 ToyRISC ISA deep dive: all 3 instruction formats with complete bit-level encoding, all 30 operations with detailed semantics and Java implementation hints, sign extension, object file format, assembly syntax, complete manual encoding example, instruction decode logic.

Part 3 Simulator architecture: complete code walkthrough of all Java classes, package dependencies, configuration system, detailed analysis of all 5 test programs (with execution traces), statistics and performance metrics, implementation checklist, common pitfalls and debugging tips, and preview of future topics (pipelining, caches, multi-cycle units).

— End of Part 3 —